

Applied NLP

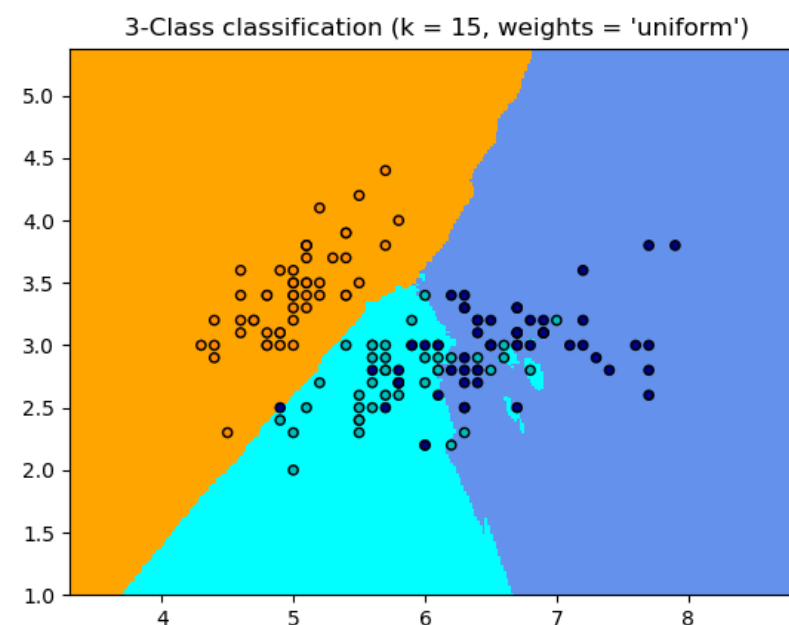
DATA 641

Lecture 6 — Traditional text classification

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly.

Example 1: k -Nearest Neighbor Classification

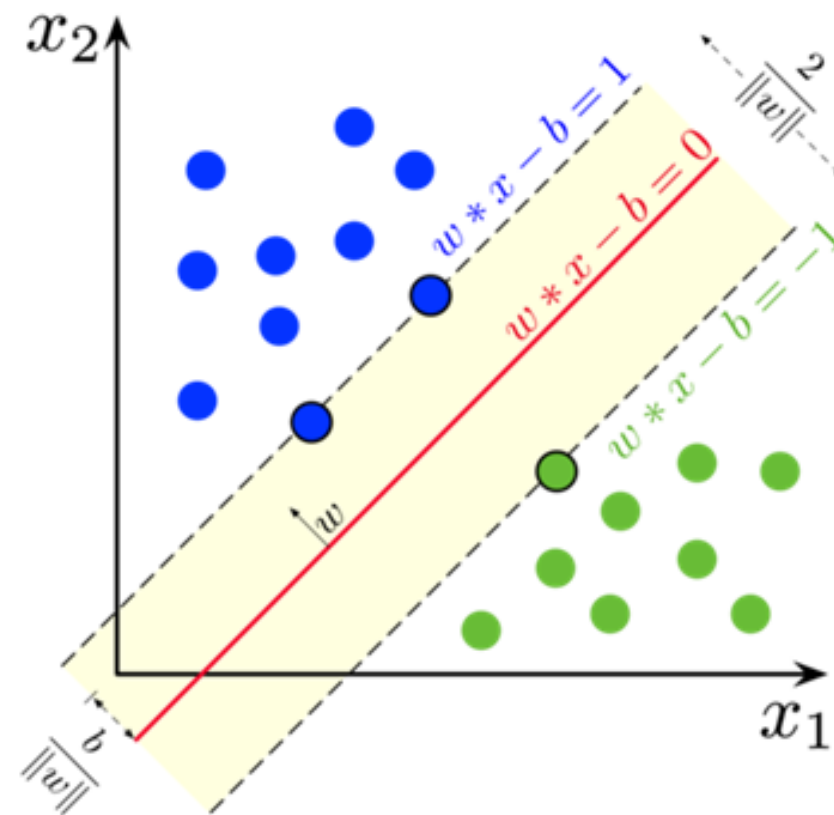
- The idea behind k -Nearest Neighbors (k -NN) classification is to assign a class label to a new data point based on the majority class of its k nearest neighbors in the feature space.
- The algorithm operates on a labeled training dataset, where each instance has a set of features and a corresponding class label. To classify a new data point, the algorithm calculates the distances between the new point and all the points in the training set. It then selects the k nearest neighbors, where k is a predefined parameter.
- The majority class label among the k nearest neighbors is assigned as the predicted class label for the new data point.



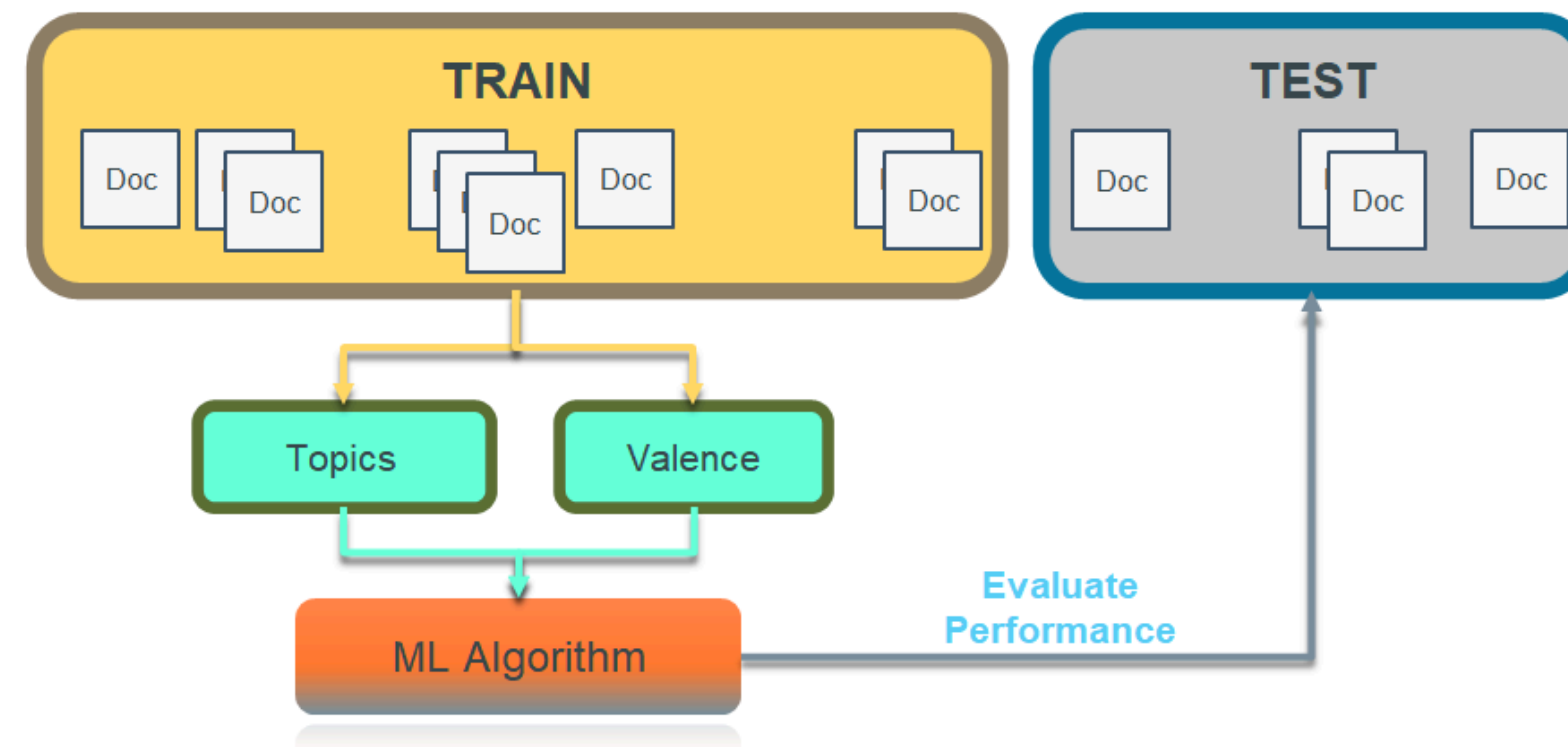
Example 2: Support Vector Machines

We are given a training dataset of n points $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$ where y_i is either 1 or -1 each indicating the class. We would like to estimate a maximum-margin hyperplane which separates points by their labels y .

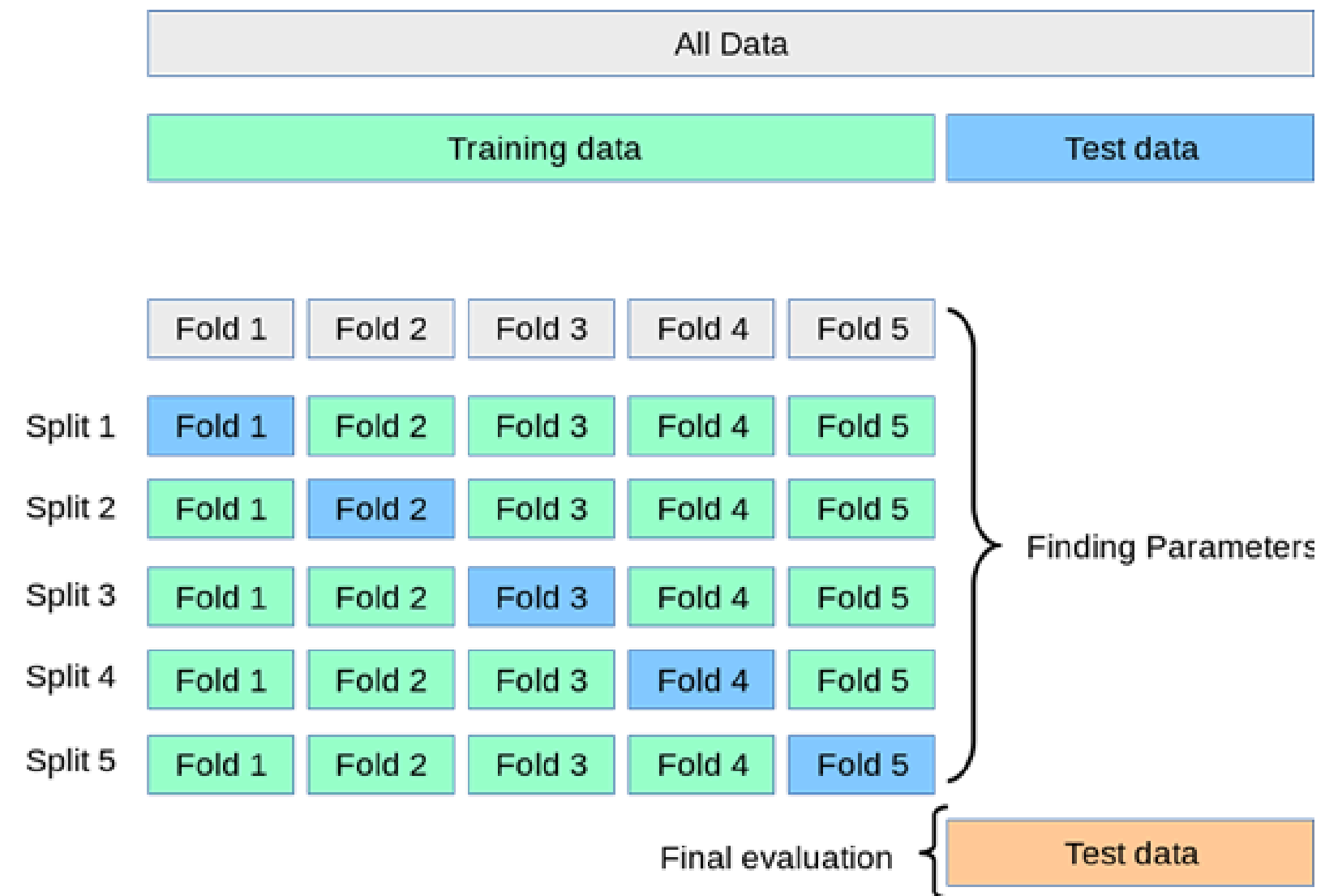
The objective is to minimize $\|\mathbf{w}\|$ subject to $y_i(\mathbf{w}\mathbf{x}_i - b) \geq 1$



Cross-Validation: In pictures



K-fold cross-validation



Parameters vs Hyper-parameters

Parameters whose
values are *learned*

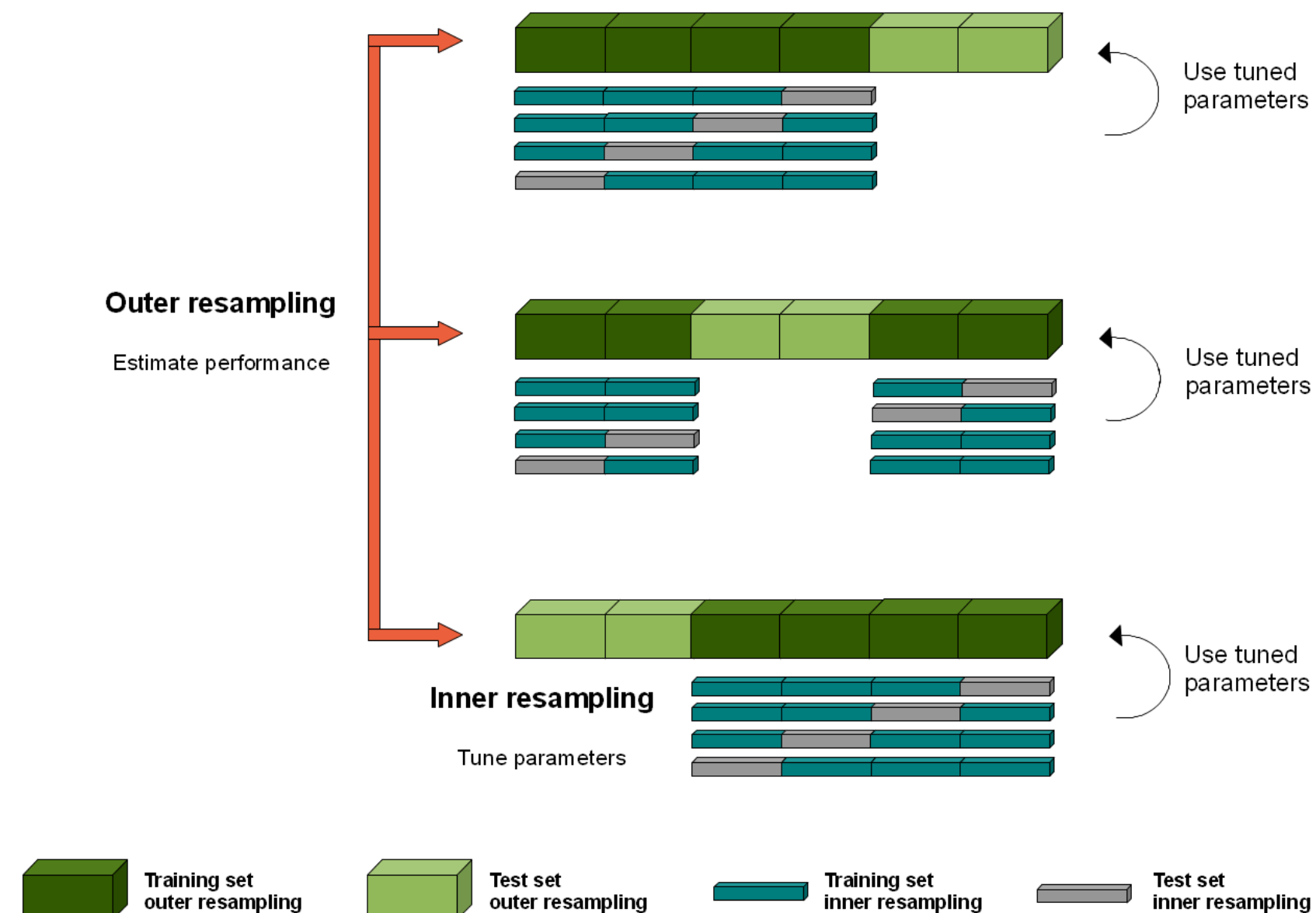
Feature	β
the	0.01
and	0.03
bravest	1.4
love	3.1
loved	1.2
genius	0.5
<i>BIAS</i>	-0.1

Hyperparameters whose
values are *chosen*

Hyperparameter	value
minimum word frequency	5
max vocab size	10000
lowercase	TRUE
regularization strength	1.0

Nested Cross-validation — Hyper-parameter optimization

Nested cross-validation combines cross-validation and hyper-parameter tuning. It is used to evaluate the performance of a machine learning algorithm and also estimates the generalization error of the underlying model and its hyper-parameter search.



Evaluating classifiers

Imagine a text sentiment classifier, where you try to predict happy or sad. The most "natural" way to evaluate such model is to use accuracy

$$\text{accuracy} = \frac{\text{number of correctly predicted}}{\text{total number of observations}} \quad (1)$$

Caveat: Only good when the number of positive/negative observations are comparable

- Accuracy is (typically) not enough. For instance in the class imbalance case which means that one class has way more samples than the other.
- Different methods to evaluate classifier performance include **precision, recall, f1 score, ROC curve**.

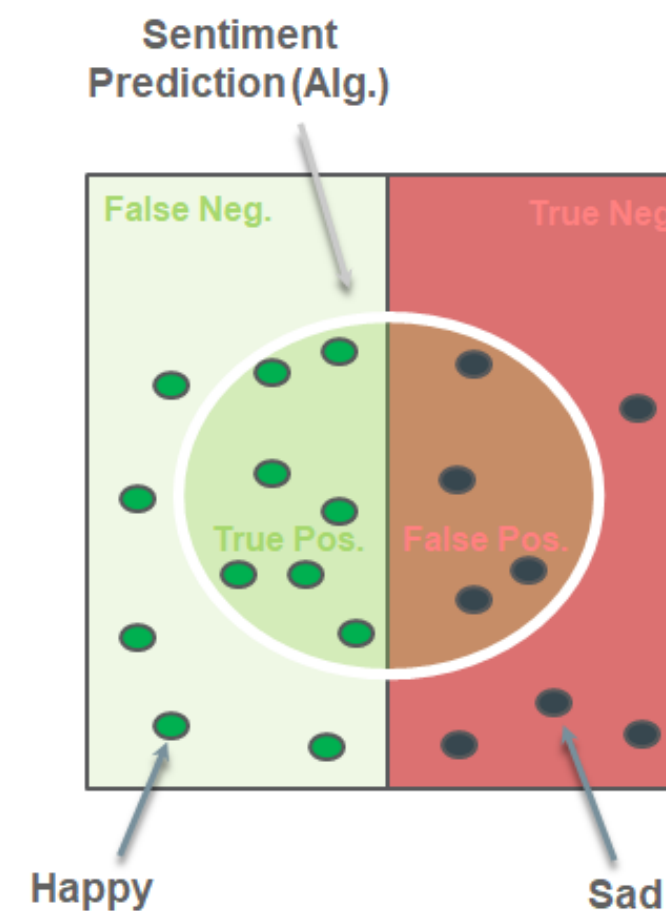
Precision, recall, f1 score, and ROC curve

Precision: How many of the predicted "happy" document are actually "happy"?

Recall: How many of the "happy" documents did we identify from the corpus?

f1 score: Harmonic mean of precision and recall

Receiver Operator Characteristic: Plots the true positive rate against the false positive rate



Evaluation of models via SkLearn

Look up:

- `sklearn.metrics.f1_score`
- `sklearn.metrics.precision_score`
- `sklearn.metrics.recall_score`
- `sklearn.metrics.roc_curve`

```
from sklearn.metrics import classification_report
y_true = [0, 1, 2, 2, 2]
y_pred = [0, 0, 2, 2, 1]
target_names = ['class 0', 'class 1', 'class 2']
print(classification_report(y_true, y_pred, target_names = target_names))
```

	precision	recall	f1-score	support
class 0	0.50	1.00	0.67	1
class 1	0.00	0.00	0.00	1
class 2	1.00	0.67	0.80	3
accuracy			0.60	5
macro avg	0.50	0.56	0.49	5
weighted avg	0.70	0.60	0.61	5